



COURSE MATERIAL

From Hardware to Interface

CMSC 125 Problem Set 3

Prepared by: Rene Andre B. Jocsing

Published: March 26, 2026

Deadline: April 20, 2026

This problem set spans the full vertical extent of the persistent storage stack: from the physical mechanisms of I/O devices and hard disk drives up through the system calls that programmers use to name, read, and modify files. Each layer was designed with knowledge of the layer below it, and each introduces abstractions that hide cost while creating new constraints. Understanding these three layers in relation to one another—not as isolated topics—is the central goal of this assignment. It requires careful reading of the primary references and substantial independent research to move beyond description toward genuine analysis.

Background

Persistent storage is the third of the three great problems of operating systems, alongside virtualization and concurrency. Unlike CPU and memory virtualization, which provide each process the illusion of a private machine, persistent storage is explicitly *shared*: files outlive the processes that create them, and multiple processes may read or write the same file simultaneously. The OS must therefore manage hardware that is orders of magnitude slower than DRAM, provide an interface that is simultaneously simple for programmers and safe for concurrent use, and do so without losing data when the machine loses power.

The three chapters assigned for this unit each address a distinct level of this problem. Arpaci-Dusseau & Arpaci-Dusseau (2023) Chapter 36 describes how the OS abstracts over raw hardware devices using interrupts, DMA, and device drivers. Chapter 37 builds a concrete model of hard disk performance—one of the most consequential performance models in systems—and derives disk scheduling algorithms from first principles. Chapter 39 presents the UNIX file system interface: the file descriptors, open file table, inodes, hard links, symbolic links, and durability guarantees that programmers encounter every time they call `open()`, `write()`, or `fsync()`.

These three chapters are not independent. The I/O device model explains *how* the OS issues a disk request. The disk performance model explains *how much that request costs* and in what order requests should be issued. And the file system interface explains *what sequence of requests* a single programmer-facing system call actually generates. An `open()` call, for instance, triggers multiple inode and directory reads, each of which descends through the driver to the physical device. Understanding the full path from system call to spinning platter—and the cost implications at every step—is what distinguishes a systems programmer from someone who merely uses POSIX.

Learning Objectives

After completing this problem set, students should be able to:

- Explain the canonical I/O device model and the trade-offs between polling, interrupts, and DMA
- Describe how device drivers provide a uniform interface and analyze the costs of that uniformity
- Model hard disk I/O time using the seek-rotation-transfer formula and reason quantitatively about sequential versus random access performance
- Analyze disk scheduling algorithms (SSTF, SCAN/C-SCAN, SPTF) in terms of what each optimizes and where each fails
- Describe the three-layer model of file access (per-process fd table, open file table, inode) and explain why the offset lives in the middle layer
- Explain the link-count model behind `unlink()`, and distinguish hard links from symbolic links structurally and practically
- Analyze the durability guarantees of `write()`, `fsync()`, and `rename()` and explain what application patterns they together enable
- Trace a file system operation from the programmer's system call down through the device driver to the physical disk, identifying where each chapter's concepts appear
- Conduct independent research using academic sources and synthesize technical concepts into a coherent argument

Essay Topic: The Anatomy of Persistent Storage

Using [Chapter 36](#), [Chapter 37](#), and [Chapter 39](#) of Arpaci-Dusseau & Arpaci-Dusseau (2023) as your primary references, along with additional academic sources, write a comprehensive essay that addresses the following four sections. Each section should constitute approximately 25% of your essay's content.

1. Managing I/O Devices (25%)

Explain how the operating system manages communication with hardware devices, using the concepts introduced in Chapter 36 as your foundation. Your discussion must address the following in a coherent, connected argument rather than as a checklist:

- Describe the **canonical device model**: the status, command, and data registers that form the basic interface between the OS and any device. Walk through the canonical polling protocol step by step, and identify precisely where its inefficiency lies. This is your baseline: polling works, but at what cost?
- Explain how **interrupts** solve the polling problem, and describe in detail what happens when a device raises one—from the device's signal to the OS resuming the waiting process. Then complicate this: under what conditions does an interrupt-based approach *introduce more overhead than it saves*? Your discussion should address at least two such conditions (e.g., fast devices, high-rate packet arrival) and explain the hybrid strategies that emerge from them.
- Explain **DMA** and why it is necessary even when interrupts are already in use. Polling eliminates one inefficiency; interrupts eliminate another; DMA eliminates a third. Clearly distinguish what each mechanism addresses and what it does not.
- Describe the role of the **device driver** in the Linux I/O stack. Explain how the driver allows the file system to remain device-agnostic, and then evaluate the cost of this abstraction: what does the OS give up by routing all disk I/O through a generic block interface? Note that over 70% of Linux kernel code consists of device drivers (Arpaci-Dusseau & Arpaci-Dusseau, 2023)—what does this figure reveal about the practical scope of I/O management?

2. Hard Disk Performance and Scheduling (25%)

Hard disk drives are one of the most consequential performance bottlenecks in computing history, and their physical characteristics have shaped decades of file system and database design. Chapter 37 provides a functional model of disk performance that remains practically relevant even as SSDs become more common. Your discussion must:

- Describe the **physical geometry** of a hard disk drive—platter, surface, spindle, track, disk head, and arm—and explain how this geometry gives rise to the three phases of every disk I/O request. State

the I/O time formula $T_{IO} = T_{seek} + T_{rotation} + T_{transfer}$ and explain what each term represents physically, not just mathematically.

- Using the **Cheetah 15K.5 and Barracuda** drive specifications from the chapter (Arpaci-Dusseau & Arpaci-Dusseau, 2023), perform the random and sequential I/O rate calculations described in Chapter 37. Show your work. Then explain, in plain terms, what the roughly 200× gap between random and sequential throughput implies for a programmer writing software that accesses disk. Why does this gap persist even as individual drive components improve? What design principle for file systems and applications does it motivate?
- Describe the **SSTF, SCAN/C-SCAN, and SPTF** disk scheduling algorithms. For each: explain what it optimizes, identify its specific failure mode or limitation, and give a scenario in which it would be the appropriate choice. Be precise about why SSTF causes starvation and exactly how SCAN prevents it without fully solving the underlying problem that SPTF addresses.
- Reflect: given that modern NVMe SSDs have no moving parts, no seek time, and negligible rotational delay, **is disk scheduling still relevant?** Identify what from Chapter 37's analysis still applies to SSDs (e.g., the sequential-versus-random performance gap, the importance of request ordering) and what becomes irrelevant. Your argument here should be supported by at least one source beyond Arpaci-Dusseau & Arpaci-Dusseau (2023).

3. The File System Interface as Persistent Abstraction (25%)

Chapter 39 describes the interface through which every programmer interacts with persistent storage. The design of this interface—what it exposes, what it hides, and what guarantees it makes—reflects decades of accumulated experience with the hardware described in Chapters 36 and 37. Your discussion must:

- Explain the **three-layer model** of file access: the per-process file descriptor table, the system-wide open file table (with its current offset field), and the on-disk inode. Trace what happens when a process calls `open()` and receives a file descriptor back. Explain why the current offset is stored in the open file table entry rather than in the per-process descriptor table or in the inode itself—this is not arbitrary; the `fork()` behavior described in Chapter 39 (Arpaci-Dusseau & Arpaci-Dusseau, 2023) reveals the design rationale. Your answer here requires understanding the offset-sharing behavior when a parent and child share a file table entry after `fork()`.
- Explain the **link-count model** that underlies `unlink()`. Trace the inode's link count across the following operations for a single file: initial creation, two `link()` calls, then three `unlink()` calls. At what point is the file's data actually freed? Why is the system call named `unlink()` rather than `delete()`? Your answer should make clear what the distinction between a directory entry and an inode is, and why that distinction makes hard links possible.
- Compare **hard links and symbolic links** along the following dimensions: what each physically is on disk, whether it can span file systems, whether it can refer to a directory, and whether a dangling reference is possible. Use these structural differences to argue: for what class of use case should a programmer choose one over the other?
- Analyze the **durability guarantees** of three related system calls: `write()`, `fsync()`, and `rename()`. Explain specifically what each one guarantees and what it does not. Then describe the write-to-temp-then-rename pattern shown in Chapter 39 (Arpaci-Dusseau & Arpaci-Dusseau, 2023) and explain why a programmer who needs atomic, durable file updates must use all three. What class of application makes this pattern necessary, and why is `write()` alone insufficient?

4. Synthesis: The Cost and Coherence of Layered Abstraction (25%)

The three preceding sections address distinct layers of the storage stack. This final section asks you to argue across all three, treating them as an integrated system rather than three separate topics. You must take a defensible position and support it; a section that merely summarizes the three chapters will receive little credit here.

- **Trace a single system call across all three layers.** Consider the operation `open("/foo/bar", O_RDONLY)` issued by a user process. Describe the chain of events from this call down to physical disk I/O and back, identifying precisely where each chapter's concepts are invoked: the device driver

and interrupt mechanism (Ch. 36), the disk seek and rotation costs (Ch. 37), and the inode and directory traversal (Ch. 39). Your goal is not to catalog all possible events, but to show how the three chapters form a coherent vertical description of a single operation.

- **Evaluate the abstraction boundary between Chapters 37 and 39.** The file system interface (Ch. 39) is designed to hide the disk geometry (Ch. 37) from the programmer. Argue: does it succeed? Identify at least two ways in which disk hardware characteristics *leak through* the file system abstraction and influence how a well-informed programmer should write their code. Consider, for example, the implication of the sequential-versus-random performance gap for how a programmer should structure file access patterns, or the implication of the write-buffer delay for how a database must call `fsync()`. Concrete examples are required.
- **Take a position on the UNIX file system interface.** Given what you now know about the hardware the interface runs on, argue whether the UNIX file system API as described in Chapter 39 (Arpaci-Dusseau & Arpaci-Dusseau, 2023) is a well-designed abstraction. Acknowledge its genuine strengths—for instance, the uniformity of naming, the simplicity of the `fd` model, the atomicity of `rename()`—and identify at least one aspect you find inadequate or misleading given the hardware realities described in Chapters 36 and 37. Your critique should be grounded in technical argument, not preference.

Research Requirements

This essay requires research beyond Arpaci-Dusseau & Arpaci-Dusseau (2023). Both of the preloaded sources are your starting point, not your finish line. Essays that cite only the two preloaded sources will not meet the research standard for this assignment.

You must:

- Cite **at least three (3) academically sound sources** using **APA 7th Edition** format
- Arpaci-Dusseau & Arpaci-Dusseau (2023) counts as one source; Ruemmler & Wilkes (1994) counts as a second—you need **at least one additional source** that you found independently, beyond what is preloaded here
- Acceptable sources include:
 - Peer-reviewed papers from conferences (ACM SIGOPS, USENIX, FAST, etc.)
 - Technical books on operating systems, storage systems, or systems programming (see: syllabus)
 - Academic journal articles on I/O, disk performance, or file system design
 - Official Linux kernel documentation on the I/O subsystem or VFS layer
- **Not acceptable:** Wikipedia, blogs, online forums, AI-generated content presented as original research
- Include a **References** section at the end following APA 7th Edition format and include in-text citations
- The References section does **not** count toward your word count

The following are suggested entry points from the chapters' own reference lists. They are freely available and directly relevant—but again, you are expected to go further:

- Ritchie & Thompson (1974)—the original UNIX paper, whose file system design is the direct ancestor of Chapter 39's interface
- McKusick et al. (1984)—the Fast File System, which introduced many interface features (long file names, symbolic links) now treated as standard
- Coffman, Klimko & Ryan (1972)—the foundational disk scheduling paper that introduced SCAN and its variants
- Swift, Bershad & Levy (2003)—on improving OS reliability by isolating device drivers, illustrating the cost of the driver abstraction from Chapter 36
- Patterson, Gibson & Katz (1988)—the RAID paper, which treats disk performance characteristics (from Chapter 37) as the problem to be solved

Strong essays will draw on sources not listed here. Use Google Scholar, the ACM Digital Library, or IEEE Xplore to find work on topics such as: the performance implications of the VFS layer, the design

of NVMe and its scheduling implications, empirical studies of file system access patterns, or the history of the inode structure. Section 4 in particular is strengthened by sources that take a critical or historical view of the UNIX interface.

Format Requirements

- Write on **one-whole sheets of yellow pad paper only**
 - Work together with your pair defined [in this sheet](#) to accomplish the essay
 - Your essay must be approximately **2000 words** (acceptable range: 1800–2200 words)
 - Essays outside this range will receive deductions
 - **CRITICAL:** You must write the **cumulative word count at the left margin of each line**
 - Example: Line 1 ends at word 8, write 8 at the left margin
 - Line 2 ends at word 17, write 17 at the left margin
 - Continue throughout the entire essay
 - Write **legibly in PRINT** (not cursive)
 - Illegible answers will receive reduced credit
 - **Minimal erasures:** If you make a mistake, neatly cross out with a **single line** (avoid correction fluid or tape!)
 - Excessive corrections leading to illegible writing will result in deductions
 - You may include hand-drawn diagrams, tables, or calculations to illustrate your points
 - Such visual aids do not add to the essay's word count but can enhance clarity
-

Submission Instructions

1. Submission must occur **in-person during the release of the final examination** to the instructor on or before **April 20, 2026 (up to 5:30PM)**
 2. All yellow pad sheets must be **stapled together** securely in the upper-left corner
 3. Write your **FULL NAMES** on the top left of the first page
 4. Write the **TOTAL WORD COUNT** on the top right of the first page
 5. Include page numbers on the bottom right of each sheet
 6. As per the syllabus, late submissions for lecture requirements will **NOT** be accepted except with university-sanctioned documentation (medical emergencies, family emergencies, etc.)
-

Academic Honesty

This is a **paired assessment** (groups of 2, or a trio in case of an odd number of students). Discussion with your peers is expected and encouraged, but the final output must come entirely from your group. You may use AI tools (ChatGPT, Claude, Deepseek, etc.) for research and to aid your understanding of complex concepts, but you must read, comprehend, and synthesize the information yourselves.

The final essay must be written by hand in your own words. As per the syllabus, direct copying from AI outputs or other sources is a serious academic offense. Violations will result in:

- Automatic failure in the course (5.0)
- Referral to the university disciplinary committee
- Possible expulsion from the university

Proper citation of all sources is mandatory. Plagiarism (presenting others' work as your own) will be prosecuted to the full extent of university policy.

Evaluation Criteria

Your essay will be evaluated on five dimensions:

- **Technical Accuracy (30%):** Correctness of OS concepts, mechanism descriptions, calculations, and the tracing of operations across layers
- **Depth of Analysis (25%):** Cross-layer synthesis, engagement with trade-offs, quality and defensibility of the position taken in Section 4
- **Research Quality (20%):** Quality and integration of academic sources beyond the primary reading, proper APA citations
- **Organization & Clarity (15%):** Logical flow within and across sections, precise technical writing, coherent transitions
- **Format Compliance (10%):** Adherence to word count, line numbering, legibility, and submission requirements

See the detailed rubric below for specific performance standards.

Tips for Success

- **Read all three chapters before writing anything:** The essay is explicitly cross-chapter. Reading only the chapter for your assigned section will produce a disjointed essay that scores poorly on Depth of Analysis.
 - **Do the disk calculations by hand first:** Section 2 requires you to work through the Cheeta/Barracuda numbers. Do this on scratch paper before writing. An essay that states conclusions without showing the arithmetic will receive partial credit only.
 - **Divide the research, not just the writing:** Each partner should explore sources for different sections independently before sharing findings and drafting together. A single partner doing all the research produces a lopsided essay. If possible, meet in person for a proper discussion.
 - **Section 4 requires a claim:** You must argue a position, not summarize. Decide what you think before you write, outline your reasoning, then write toward your conclusion. “The UNIX interface is mostly good, but...” is a position. “The UNIX interface has both strengths and weaknesses” is not.
 - **The trace in Section 4 is a test of integration:** If you cannot describe where Chapter 36 appears in the path of an `open()` call, you have not yet understood how the three layers relate. Work through this mentally before writing.
 - **Count as you write:** Tracking cumulative word count per line from the start avoids painful recounting at the end. Practice on a page beforehand if needed.
 - **Write the essay together:** With two authors, it is tempting to divide the four sections and staple them. Resist this. The essay must read as one unified argument, with transitions between sections that acknowledge what the previous section established. Essays that are merely stapled together will score poorly across all rubrics.
-

Common Pitfalls to Avoid

- **Treating sections as independent summaries:** Each section should build on the previous one. Section 2 assumes the device model from Section 1; Section 3 assumes the performance model from Section 2. Write accordingly.
- **Skipping the calculations:** The quantitative comparison of random versus sequential I/O is a required element of Section 2. Qualitative descriptions of the gap without numbers will receive reduced credit.
- **Confusing `lseek()` with a disk seek:** Chapter 39 is explicit about this distinction. Conflating the two is a common error that signals insufficient reading.
- **Describing hard and soft links without structural grounding:** Many students describe the behavior of links without explaining what each one physically is (a directory entry vs. a file whose contents are a pathname). Structural grounding is required.

- **A Section 4 that summarizes instead of argues:** This section is where depth of analysis is most directly assessed. If it reads like a conclusion paragraph that recaps Sections 1–3, it will score poorly.
 - **Citing only OSTEP and Ruemmler & Wilkes:** This is explicitly insufficient. You need at least one additional independently found source, and strong essays will have two or more.
 - **Weak source integration:** Sources cited only in the References section without being woven into the argument earn no Research Quality credit. Cite sources where the idea they support appears.
 - **Format violations:** An essay that is 100 words under or over the range, or that has no cumulative line counts, will lose Format Compliance points regardless of its technical quality.
-

Why Handwritten?

You may wonder why this assignment requires handwritten submission in an age of word processors and digital documents. This is a deliberate pedagogical decision designed to maximize your learning outcomes.

The AI Reality: Modern AI tools can generate technically accurate essays on operating systems concepts. However, this course aims to develop *your* understanding, not the AI's. The handwritten requirement creates a necessary friction point in the workflow.

Forced Engagement: If you use AI assistance for research or drafting, you cannot simply copy-paste the output. You must physically transcribe every word by hand. This process forces you to:

- **Read every sentence carefully:** You cannot transcribe text you haven't read
- **Process the information:** Handwriting is slower than typing, giving your brain time to process concepts as you write them
- **Identify gaps in understanding:** When you write something you don't understand, it becomes immediately apparent. You'll naturally pause, reread, and seek clarification
- **Engage with technical details:** Transcribing formulas, calculations, and system call sequences requires active attention to detail

Cognitive Benefits: Research in educational psychology consistently demonstrates that handwriting promotes deeper cognitive processing than typing. The act of forming letters, combined with the slower pace, enhances memory encoding and conceptual understanding. When you handwrite technical content, you are more likely to recall it during examinations and apply it in future coursework.

Academic Integrity: This format also serves an integrity function. While AI assistance is permitted for research and understanding, the final product must demonstrate *your* comprehension. A handwritten essay in your own words ensures the work is authentically yours.

Professional Preparation: In technical interviews and graduate school qualifying exams, you will often work through problems by hand on whiteboards or paper. This assignment provides practice in organizing complex technical arguments without digital aid—a skill that does not come naturally to those who have only ever typed.

The goal is not to make your life difficult, but to ensure that the time you invest in this assignment translates into genuine learning. The concepts in these three chapters will appear on the final examination. Students who have handwritten them tend to recall them more reliably than those who merely read them.

References

See the problem set guide for the list of references used.

Essay Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Technical Accuracy (30%)	All OS concepts correct; mechanisms described precisely; calculations shown correctly; examples accurate; no conceptual errors.	Core concepts correct; minor inaccuracies in details; calculation errors or missing steps; examples mostly accurate.	Some correct concepts; significant errors in mechanisms; failed calculations; confused examples.	Fundamental misunderstanding; incorrect mechanisms throughout; no accurate calculations or examples.
Depth of Analysis (25%)	Synthesizes concepts across sections; explores tradeoffs deeply; concrete examples throughout; anticipates edge cases; critical thinking evident; goes beyond surface description.	Good analysis; explores some tradeoffs; provides adequate examples; shows solid understanding of implications and relationships.	Mostly descriptive; limited analysis; few examples; surface-level understanding; minimal exploration of tradeoffs.	Purely descriptive; no analysis; regurgitates definitions; fails to address prompt requirements; no critical thinking.
Research Quality (20%)	3+ high-quality academic sources; meaningful integration into argument; perfect APA citations; sources directly support claims; demonstrates independent research.	3 acceptable sources; generally correct APA format; relevant sources with adequate integration; some independent research evident.	Minimum sources met; citation errors; weak integration; sources tangentially relevant or poorly utilized.	Missing required citations; non-academic sources only; no APA format; plagiarism detected; no evidence of research.
Organization & Clarity (15%)	Logical flow between sections; clear topic sentences; smooth transitions; precise technical writing; arguments build coherently; easy to follow throughout.	Generally organized; mostly clear explanations; adequate transitions; readable throughout with minor structural issues.	Weak organization; unclear explanations; choppy transitions; difficult to follow in places; some technical writing issues.	Incoherent structure; incomprehensible explanations; no clear argument thread; impossible to follow.
Format Compliance (10%)	Word count within range; cumulative line counts correct throughout; legible print handwriting; minimal/no erasures; proper References section; all requirements met.	Minor word count deviation (<50 words); line counts mostly correct with few gaps; mostly legible; some erasures but readable; References present with minor errors.	Significant word count deviation (50-100 words); many missing line counts; partially illegible; excessive erasures; incomplete References.	Word count requirements ignored; missing line counts; illegible handwriting; violates format requirements; no References; critical submission violations.